

Architecture & Design Breakdown

Program Counter

The Program Counter (PC) keeps track of the instruction currently being executed. Under normal operation, the PC increments on each clock cycle, so the processor continues to the next instruction in instruction memory.

When a jump, subroutine call, or successful conditional branch occurs, the PC is instead loaded with a new address, and execution continues normally from that location.

Since the program counter is 8 bits wide, the PC can only address up to 256 instruction locations (0-255).

Instruction Memory

The instruction memory holds the 16-bit instructions in which the program counter chooses which instruction is to be executed. The contents of instruction memory are generated and translated from assembly code using the Python assembler. The assembler translates from a human-readable assembly instruction into a 16-bit machine code format defined by the custom ISA.

Instruction Decoder

The instruction decoder separates the 16-bit instruction into the fields required by the rest of the CPU. Depending on the instruction, these fields may include:

Opcode, Destination Register, Source Register, Immediate Values, Addresses, and more.

The most important is the opcode, which is sent to the control unit to decide what the CPU is to perform.

The Control Unit

The control unit determines how the processor should respond to the instruction currently being executed. It receives the decoded opcode along with ALU flags (carry, negative, and zero flags) and generates the control signals based on what it reads and enables the corresponding actions throughout the CPU.

The control signals may determine:

- Which ALU operation is performed
- Whether a register is written
- Whether memory is read or written
- Which value is written back to the register file
- Whether a branch is taken
- Whether the Program Counter changes
- Whether a stack operation occurs
- Whether execution is halted
- Whether the active memory bank changes

The control unit acts as the brain of the system and controls the flow of information. For an instruction such as “ADD R3, R1, R2”:

The control unit identifies the ADD opcode and configures the datapath so that

1. R1 and R2 are read from the Register File.
2. The ALU performs addition
3. The resulting value is routed back to the Register File.
4. Register R3 is enabled for writing.

For a conditional branch instruction, the Control Unit also examines the ALU flags to determine whether the Program Counter should jump to the branch target,

Register File

The processor contains eight general-purpose 8-bit registers: R0 - R7.

The register file provides temporary high-speed storage directly inside the CPU.

Registers are commonly used for arithmetic operands, operation results, loop counters, manually loaded values, memory values, and as pointers.

The instruction decoder selects which registers should be read based on the register fields contained in the current instruction. Because the registers are located directly within the CPU datapath, they provide faster and simpler access than storing every temporary value in data memory.

Arithmetic Logic Unit

The Arithmetic Logic Unit (ALU) performs the mathematical and logical operations needed by the CPU. Operations supported by the ALU include:

- ADD
- SUB
- AND
- OR
- XOR
- NOT
- INC
- DEC

The Control Unit selects the required ALU operation based on the current instruction opcode.

For example:

“SUB R3, R1, R2”

Causes the registers R1 and R2 to output their contents, and the ALU will subtract and the result is written back into R3.

The ALU generates three important flags:

Zero Flag is set when the result equals zero and is used for conditional jump if zero (JZ) and jump if not zero (JNZ) instructions.

Carry / Borrow Flag indicates an unsigned carry during addition or a borrow during subtraction. It can be used for conditional branching jump if carry (JC)

Negative Flag reflects the most significant bit of the ALU result and indicates whether the result represents a negative value when interpreted as a signed 8-bit number.

Data Memory

The Data Memory stores program data separately from the instruction memory. Each memory location stores an 8-bit value, matching the width of the CPU datapath and registers.

The processor can interact with data memory using load and store instructions. For example, STORE R1, 50 writes the contents of R1 into data-memory address 50 and LOAD R2, 50 reads the value stored at address 50 and writes it into R2.

Unlike instruction memory, which contains executable machine instructions, data memory stores values used by programs while they run.

Memory Banking

The CPU uses memory banking to expand the amount of available data memory while preserving the existing 8-bit memory-address format.

A separate bank register determines which memory bank is currently active and can cycle between banks 0 to 3. A global section is preserved between all banks for memory-mapped I/O and stack values. Therefore, it can be accessed no matter the bank number. The four banks multiply the number of general memory locations by four.

Memory Mapped I/O

The computer uses memory-mapped I/O to connect the processor to hardware peripherals. Instead of introducing separate input and output instructions, specific data-memory slots are reserved for peripheral input and outputs.

The CPU therefore can communicate with peripherals using only normal LOAD and STORE instructions.

For example, LOAD R1, 248 takes a received byte from UART and stores it into register R1.

Memory-mapped peripherals in the design include:

- FPGA LEDs
- FPGA switches
- FPGA push buttons
- FPGA seven-segment display
- UART

This helps keep the ISA simpler because software interacts with RAM and peripherals through the same memory interface.

Stack Pointer and Hardware Stack

The CPU contains a Stack Pointer (SP) used to manage the hardware stack. The stack operates using Last-In, First-Out (LIFO) behaviour, meaning the most recently stored value is the first value removed. The CPU provides explicit stack operations.

PUSH stores a value at the current stack location and decrements the stack pointer.

POP retrieves the most recent stored value and increments the stack pointer.

The stack is stored within a reserved portion of global data memory so that it remains available regardless of the active memory bank.

The stack enables reusable and nested subroutines, which helps reduce repeated code within the 256-instruction program space.

CALL pushes the return address to the stack and jumps to the given address.

RET pops the saved return address and loads it back to the program counter. The program then executes immediately after the original CALL.

Because of the LIFO behaviour, it allows for nested subroutines.

UART - Universal Asynchronous Receiver Transmitter

The computer includes a bidirectional UART interface for communication with an external PC. UART does not run off a shared clock and therefore runs between a shared baud rate between two devices. The baud rate chosen for this CPU is 115200 baud (bits per second); therefore, in order to properly send and receive bytes, the serial terminal used must also be set to 115200 baud.

UART is an asynchronous, so only one bit can be sent at a time, and the format used here is 8-N-1, which stands for 8 data bits, no parity bit, and 1 stop bit.

UART is integrated into the memory-mapped I/O system so the CPU can interact with it using normal load and store instructions. UART contains separate transmit and receive hardware.

UART Receive

Incoming serial bits from the PC are sampled by the UART receiver and reconstructed into 8-bit bytes. Completed bytes are placed into a receive FIFO (First-In, First-Out). The FIFO temporarily stores received characters similar to a queue until the processor is ready to process them. This prevents the CPU from needing to read each byte at the exact instant it arrives.

Read and write positions are tracked internally so incoming bytes can be added while previously received bytes wait to be processed. If the FIFO becomes full and another byte arrives, the UART subsystem detects a receive overflow condition instead of silently replacing unread data.

UART Transmit

For transmitted data, the CPU writes an 8-bit value to the UART transmit interface. The UART transmitter converts the byte into the required serial format and sends the bits sequentially to the connected PC.

The CPU can check UART status information to determine whether:

- A received byte is available
- The transmitter is ready
- The receive FIFO contains data
- An overflow condition has occurred

This UART interface allows assembly programs to operate interactively through a serial terminal.

Seven-Segment Display

The Basys 3 seven-segment display provides a built-in visual output device for the computer. CPU-generated values can be sent to the display hardware through the memory-mapped I/O interface. The display logic converts values into the segment patterns required to illuminate the appropriate digits on the FPGA board. This allows programs such as the FPGA reaction time game to display numerical results without requiring an external computer.

Python Assembler

The Python Assembler provides the software interface for programming the custom CPU. It allows programs to be written using readable assembly instructions rather than manually constructing 16-bit binary machine code.

For each instruction, the assembler identifies the mnemonic and operands, determines the corresponding opcode and instruction format, validates the supplied values, and places each field into its required position within the 16-bit instruction.

For an instruction such as LOAD R2, 92 the assembler determines:

- The opcode associated with LOAD
- The binary encoding of register R2
- The 8-bit representation of memory address 92
- The correct bit positions for each field in the LOAD instruction format.

These values are then combined to produce the final 16-bit machine instruction understood by the CPU.